User Stories

1. User Manual Page Update

UMP-1 Audit manual against live app

- As a maintainer, I want to audit the manual against the live app, so that gaps are identified.
- As a maintainer, I want to compare manual content page by page against current app behaviour, so that outdated or missing instructions can be found before rewriting.

UMP-2 Updating outdated manual sections

- As a user, I want up-to-date manual sections, so that instructions match what I actually see.
- As a user, I want screenshots and instructions for the Goal List, Hierarchy, and Model View panes to be updated, so that I can follow the manual without being confused by old interface details.

UMP-3 Documenting feedback and status tracking

- As a user, I want the manual to cover feedback and status tracking, so that I know how to use those features.
- As a user, I want manual sections for the new Feedback System and Track Status features once built, so that I can understand the full review workflow.

UMP-4 Documenting version history

- As a user, I want the manual to explain how to save and restore versions, so that I understand the feature.
- As a user, I want the manual to cover checkpoints, restoring, and renaming saved versions, so that I can manage model history safely.

2. Guided tour / driver.js onboarding

GT-1 Evaluating driver.js integration

- As a developer, I want to evaluate driver.js integration, so that we know it is compatible with our stack.
- As a developer, I want to prototype adding driver.js into the React app and confirm no conflicts with mxGraph or existing UI libraries, so that the guided tour can be added safely.

GT-2 Onboarding tour for new users

- As a new user, I want a guided onboarding tour, so that I understand the three panes quickly.
- As a new user, I want a step-by-step walkthrough covering the Goal List, Hierarchy, and Model View panes, so that I can start modelling without external instruction.

GT-3 Contextual tooltips for key actions

- As a new user, I want contextual tooltips on less obvious actions, so that I do not get stuck.
- As a new user, I want driver.js highlights for actions like dragging goals into clusters, so that I can learn important interactions in context.

3. Model Version History

VH-1 Auto-save version checkpoint on Save

- As a user, I want the app to save a version checkpoint automatically when I click Save, so that I do not lose track of past states.
- As a user, I want each Save to store a timestamped snapshot in Version History, so that I can recover previous model states without keeping separate files.

VH-2 Manually save a named checkpoint

- As a user, I want to manually save a named checkpoint, so that I can mark meaningful points such as before client review.
- As a user, I want the Version History panel to provide an input and button for saving a custom-labelled checkpoint anytime, so that important versions are easy to recognise later.

VH-3 View list of saved versions

- As a user, I want to view a list of saved versions with timestamps, so that I can browse the model's history.
- As a user, I want the Version History modal to list all saved versions newest first, so that the most recent checkpoints are easy to find.

VH-4 Restore a previous version

- As a user, I want to restore a previous version, so that I can go back to an earlier state of the model.
- As a user, I want restoring to replace the current model only after a confirmation prompt, so that I do not overwrite current work accidentally.

VH-5 Rename or delete a saved version

- As a user, I want to rename or delete a saved version, so that I can keep my version list organised.
- As a user, I want each version entry to support inline rename and delete, so that I can manage saved checkpoints directly from the list.

4. Testing and QA

QA-1 Refactor SectionPanel for testability

- As a maintainer, I want SectionPanel to be refactored into smaller responsibilities, so that the code is easier to test and maintain.
- As a maintainer, I want layout logic extracted into useLayoutManager and drag-and-drop plus error handling extracted into useDragDrop, so that SectionPanel is responsible only for JSX rendering.
- As a maintainer, I want corresponding tests for the extracted layout and drag-and-drop logic, so that regressions can be caught earlier.

Source

- 15 Issues

User Stories 1

1. Client-originated requirements — Feedback

FB-1 Entering staff mode

- As a reviewer, I want to switch into staff mode from the welcome screen without changing what Create and Open do, so that I can start a review without disrupting the normal workflow.
- As a reviewer, I want to confirm I am teaching staff and type my name once per browser tab, so that I am not asked again for every file I open in the same session.
- As a reviewer, I want my session to end automatically on refresh or tab close, so that a shared or public computer never stays signed in as me.

FB-2 Writing and managing feedback

- As a reviewer, I want to add a new comment box with one click, so that I can capture feedback as fast as I read the model.
- As a reviewer, I want each comment stamped with my name and the date automatically, so that a student always knows who reviewed their work and when.
- As a reviewer, I want to edit or delete a comment I have written, so that I can fix or retract feedback before saving.
- As a reviewer, I want to be stopped at twenty comments on one model, so that feedback stays readable and every comment keeps a distinct place in the layout.

FB-3 Linking feedback to goals

- As a reviewer, I want to arm a comment and then click a goal, in the hierarchy or on the rendered diagram, to link them, so that my feedback points at something concrete instead of a vague area of the model.
- As a reviewer, I want to click an already-linked goal again to unlink it, so that I can correct a mis-click without deleting and rewriting the whole comment.
- As a reviewer, I want to leave linking mode with Esc or the same button, so that I never get stuck in a mode I have forgotten I am in.

FB-4 Reading feedback

- As a reviewer or student, I want to select one comment and see only its arrows, so that I can tell exactly which goals it is about when several comments are on screen at once.
- As a student, I want to open my model and immediately see whether it has been reviewed, so that I know there is feedback worth reading.
- As a student, I want the feedback panel to be read-only when I open it, so that I cannot accidentally edit or delete my tutor's comments.

- As a student, I want a comment that references a goal I have since deleted to still display clearly, as (deleted goal), so that I am not confused by a silently broken link.

FB-5 Saving and persistence

- As a reviewer, I want my comments saved inside the same model file the student already uses, so that there is no separate system to manage or lose track of.
- As a reviewer, I want a file I have never opened in staff mode to be distinguishable from one I have reviewed and left empty, so that "not yet marked" is never confused with "marked, no notes."
- As a user with existing model files, I want those files to keep working, so that adopting the new version costs me nothing.

2. Team-proposed requirements — Adopted

DS-1 Distinguishing the primary action buttons *(partially adopted)*

- As a user, I want the primary actions to be distinguishable at a glance, so that I do not trigger a destructive action by mistake.

DS-2 Indicating the drop target during a drag *(adopted)*

- As a user rearranging the hierarchy, I want to see where a dragged node will land, so that I do not reparent it accidentally.

3. Team-proposed requirements — In progress

VH-1 Model version history

- As a user, I want a checkpoint saved automatically whenever I save the model, so that I can go back to an earlier state without having kept copies myself.
- As a user, I want to save a named checkpoint deliberately, so that I can mark a point I know I may want to return to.
- As a user, I want to see the list of saved versions and restore one, so that I can recover from a change I regret.
- As a user, I want to rename or delete a saved version, so that the list stays meaningful as it grows.

GT-1 Guided tour for first-time users

- As a first-time user, I want a walkthrough of the main interface, so that I can start work without external instruction.

4. Deferred / Not adopted

GR-1 Grading *(withdrawn from feedback, deferred)*

- As a reviewer, I want to record a mark alongside my comments, so that the student receives the assessment and its justification together.

ST-1 Feedback status tracking *(not adopted)*

- As a subject coordinator, I want to see which models have been reviewed, so that I can identify students still waiting.